Исполнение программы на RISC-V. Регистры и память.

Содержание

Часть 1: Память и базовые инструкции

- 1. Регистры и память
- 2. Порядок байт в памяти
- 3. Почему RISC-V — load/store архитектура
- 4. Адресация памяти и вычисление адреса
- 5. Базовые инструкции работы с данными
- 6. Псевдоинструкции
- 7. Компиляция простых выражений
- 8. Как программа представлена в памяти
- 9. Линейное выполнение программы и счётчик команд
- 10. Условные переходы
- 11. Безусловный переход

Содержание (продолжение)

Часть 2: Процедуры, стек и вызовы

- 12. Циклы в RISC-V
- 13. Введение в процедуры
- 14. Инструкции для вызова процедур
- 14.1. Возврат результатов функций
- 14.2. Вложенные вызовы и сохранение адреса возврата
- 15. Регистры в RISC-V: роли и сохранение
- 15.0. Зачем нужна система ролей и сохранения?
- 16. Передача аргументов функции
- 17. Классификация регистров: временные vs сохраняемые
- 18. Стек (Stack)
- 19. Работа со стеком в RISC-V
- 20. Локальные переменные на стеке
- 21. Соглашение о вызовах (Calling Convention) в RISC-V
- 22. Пример: функция с сохранением регистров
- 23. Пример: функция с временными регистрами
- 24. Вложенные вызовы и сохранение адреса возврата
- 25. Передача больших структур
- 26. Когда избежать указателей
- 27. Карта памяти программы
- 28. Карта памяти RISC-V
- 29. Структура ELF файла
- 30. Процесс компиляции

- 26. Когда избежать указателей
- 27. Карта памяти программы
- 28. Карта памяти RISC-V
- 29. Структура ELF файла
- 30. Процесс компиляции

1. Регистры и память

1.1. Регистры

Регистры общего назначения — это небольшой набор "быстрых" ячеек хранения внутри процессора.

В RV32I их 32:

- `x0`, `x1`, …, `x31`.

Некоторые из них имеют удобные ABI-имена:

- `zero` = `x0`
- `ra` = `x1`
- `sp` = `x2`
- `a0` – `a7` — аргументы и результаты функций
- `t0` – `t6` — временные регистры
- `s0` – `s11` — сохраняемые регистры

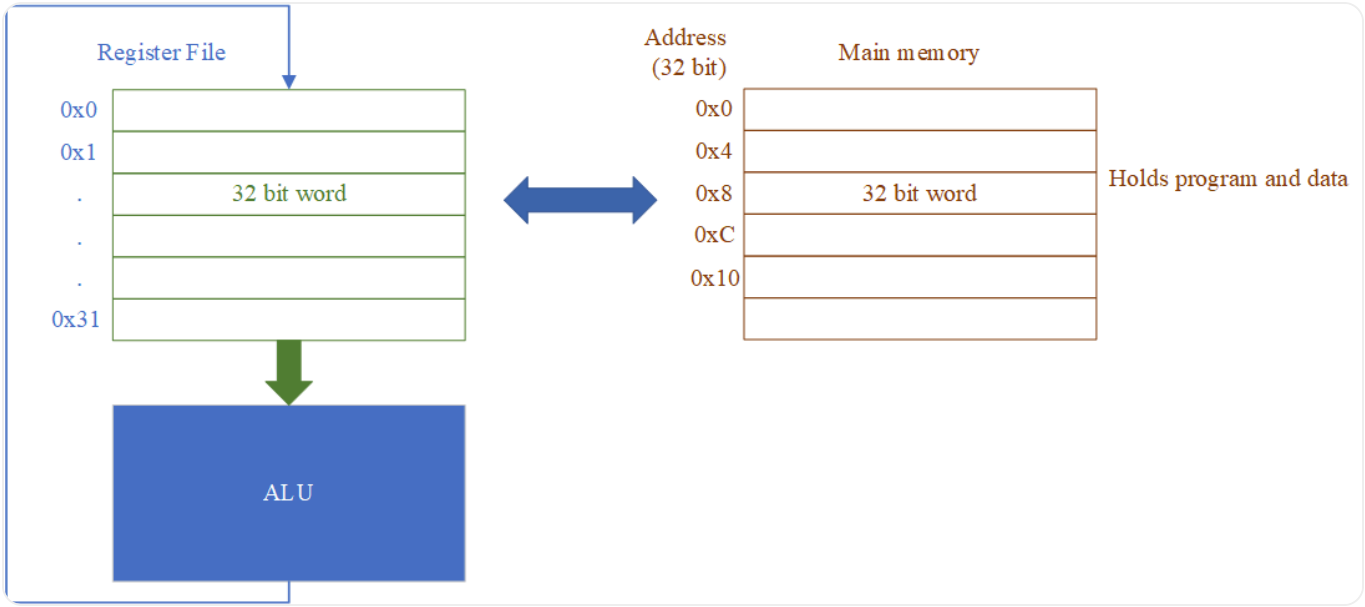
1.2. Память

Память — это крупное хранилище данных и инструкций. Она медленнее регистров, но позволяет хранить значительно больше информации.

Внутри программы в памяти могут лежать:

- код программы;
- глобальные переменные;
- массивы;
- стек;
- временные данные.

1.3. Регистры и память



- Регистровый файл – 32 регистра общего назначения
- Каждый регистр – 32 бита
- Регистр `x0` – аппаратно равен 0
- Каждая ячейка памяти – 32 бита (1 слово)
- Побайтовая адресация памяти
- Адрес 32 бита. Может быть адресовано `2^32` байт

2. Порядок байт в памяти

2.1. Little-endian

```
Word address:   0x00              0x04                  0x08                0x0C
                |                   |                     |                   |
Byte address:   00   01   02   03   04   05   06   07   08   09   0A   0B   0C   0D   0E   0F
                +----+----+----+----+----+----+----+----+----+----+----+----+----+----+----+----+
Memory byte:    | 44 | 33 | 22 | 11 | 88 | 77 | 66 | 55 | CC | BB | AA | 99 | 00 | FF | EE | DD |
                +----+----+----+----+----+----+----+----+----+----+----+----+----+----+----+----+
                  ^              ^     ^              ^     ^              ^     ^              ^
                 LSB            MSB   LSB            MSB   LSB            MSB   LSB            MSB

Words:          W0 = 0x11223344        W1 = 0x55667788        W2 = 0x99AABBCC        W3 = 0xDDEEFF00
```

2.2. Big-endian

```
Word address:   0x00              0x04                  0x08                0x0C
                |                   |                     |                   |
Byte address:   00   01   02   03   04   05   06   07   08   09   0A   0B   0C   0D   0E   0F
                +----+----+----+----+----+----+----+----+----+----+----+----+----+----+----+----+
Memory byte:    | 11 | 22 | 33 | 44 | 55 | 66 | 77 | 88 | 99 | AA | BB | CC | DD | EE | FF | 00 |
                +----+----+----+----+----+----+----+----+----+----+----+----+----+----+----+----+
                  ^              ^     ^              ^     ^              ^     ^              ^
                 MSB            LSB   MSB            LSB   MSB            LSB   MSB            LSB

Words:          W0 = 0x11223344        W1 = 0x55667788        W2 = 0x99AABBCC        W3 = 0xDDEEFF00
```

3. Почему RISC-V — load/store архитектура

- чтение из памяти выполняется отдельными инструкциями `load`;
- запись в память выполняется отдельными инструкциями `store`;
- арифметика и логика выполняются только над регистрами.

Нельзя выполнить:

```
MEM[A] = MEM[B] + MEM[C]
```

одной машинной инструкцией.

Необходимо разложить на шаги:

```
lw t0, 0(s1)
lw t1, 0(s2)
add t2, t0, t1
sw t2, 0(s3)
```

4. Адресация памяти и вычисление адреса

Чтобы прочитать или записать данные в память, нужно указать адрес.

В RISC-V для инструкций загрузки и сохранения широко используется форма:

```
lw rd, imm(rs1)
sw rs2, imm(rs1)
```

Здесь адрес вычисляется как:

```
effective_address = rs1 + imm
```

где:

- `rs1` — базовый адрес;
- `imm` — смещение.

4.1. Пример

```
lw t0, 8(sp)
```

- берется значение регистра `sp`;
- к нему прибавляется 8;
- по полученному адресу прочитать слово из памяти;
- записать его в `t0`.

Такая адресация естественна для:

- доступа к локальным переменным через `sp`;
- доступа к элементам структур;
- доступа к элементам массива;
- доступа к сохранённым регистрам в стеке.

4.2. Связь с datapath

На аппаратном уровне всё очень прозрачно:

1. значение `rs1` приходит из регистрового файла;
2. immediate извлекается и расширяется;
3. ALU складывает `rs1 + imm`;
4. полученный адрес идёт в память или в LSU.

5. Базовые инструкции работы с данными

5.1. Загрузка из памяти в RV32I

- `lw` — загрузить слово;
- `lh` — загрузить полуслово со знаковым расширением;
- `lhu` — загрузить полуслово с нулевым расширением;
- `lb` — загрузить байт со знаковым расширением;
- `lbu` — загрузить байт с нулевым расширением.

Пример:

```
lw t0, 0(s0)
```

5.2. Сохранение в память RV32I

- `sw` — сохранить слово;
- `sh` — сохранить полуслово;
- `sb` — сохранить байт.

Пример:

```
sw t0, 12(sp)
```

6. Псевдоинструкции

Команды, которые есть в ассемблере, но их нет как отдельных инструкций ISA, называются **псевдоинструкциями**.

Ассемблер принимает их для удобства, а затем разворачивает в одну или несколько настоящих инструкций.

Примеры

Псевдоинструкция	Разворачивается в	Описание
`mv t0, t1`	`addi t0, t1, 0`	копирование регистра
`nop`	`addi x0, x0, 0`	ничего не делает
`j loop`	`jal x0, loop`	безусловный переход
`ret`	`jalr x0, 0(ra)`	возврат из функции
`li t0, 5`	`addi t0, x0, 5`	загрузка малой константы
`li t0, big_imm`	`lui` + `addi`	загрузка большой константы

7. Компиляция простых выражений

Компилятор раскладывает выражение на простые операции: загрузка, вычисление, сохранение.

С-код	RISC-V
`c = a + b;`	`add t0, t1, t2`
`y = a + 5;`	`addi t0, t1, 5`
`d = (a + b) - c;`	`add t0, t1, t2` `sub t3, t0, t4`

`c = a + b;`
(`a`, `b`, `c` в памяти)
	`lw t0, 0(s0)`
	`lw t1, 4(s0)`
	`add t2, t0, t1`
	`sw t2, 8(s0)`

если данные в памяти — нужны `lw/sw`

арифметика выполняется **над регистрами**, а не над памятью

одно и то же выражение может компилироваться по-разному в зависимости от того, где лежат операнды:

- в регистрах
- в стеке
- в памяти

компилятор старается держать данные в регистрах, потому что обращение к памяти дороже

8. Как программа представлена в памяти

```
+----------------------+----------------------+-------------------------------------------------------------+
| 1) Assembly          | 2) Machine code      | 3) Code in memory (little-endian)                           |
+----------------------+----------------------+-------------------------------------------------------------+
| start:               |                      | PC        Addr     Bytes              Instruction              |
|   addi t0, x0, 5     | 0x00500293           | ->        0x0000   93 02 50 00        addi t0, x0, 5           |
|   addi t1, x0, 7     | 0x00700313           |           0x0004   13 03 70 00        addi t1, x0, 7           |
|   add  t2, t0, t1    | 0x006283B3           |           0x0008   B3 83 62 00        add  t2, t0, t1          |
|   sw   t2, 0(s0)     | 0x00742023           |           0x000C   23 20 74 00        sw   t2, 0(s0)           |
+----------------------+----------------------+-------------------------------------------------------------+

Example:
0x00500293  ->  93 02 50 00   in memory
               ^ lowest address
Execution:
PC_next = PC + 4
PC = 0x0000 -> 0x0004 -> 0x0008 -> 0x000C -> 0x0010

Rule:
PC_next = PC + 4
```

8. Как программа представлена в памяти (продолжение)

Обычно память разделяют на логические области:

- `.text` — машинный код программы;
- `.data` — инициализированные глобальные данные;
- `.bss` — неинициализированные глобальные данные;
- `stack` — стек;
- иногда отдельно обсуждают `heap` — динамическую память.

Упрощённая схема

```
+-----------------------+
|        .text          |  код программы
+-----------------------+
|        .data          |  глобальные переменные
+-----------------------+
|        .bss           |  глобальные нули/буферы
+-----------------------+
|         ...           |
+-----------------------+
|       stack           |  локальные данные, сохранённые регистры
|      растёт вниз      |
+-----------------------+
```

8. Как программа представлена в памяти (продолжение 2)

`.text`

В секции `.text` находятся инструкции. Именно туда указывает `PC`.

`.data`

Например:

```
int g = 10;
int arr[4] = {1, 2, 3, 4};
```

Эти данные должны быть где-то размещены в памяти, чтобы программа могла к ним обращаться.

`.stack`

Когда функция начинает выполняться, она часто выделяет место на стеке для:

- локальных переменных;
- сохранённых регистров;
- адреса возврата;
- временных данных.

9. Линейное выполнение программы и счётчик команд

Инструкции в программе обычно исполняются последовательно.

За это отвечает **счётчик команд** — `PC` (Program Counter), который хранит адрес текущей инструкции.

В простейшем случае после выполнения инструкции:

```
PC = PC + 4
```

так как в RV32I инструкции имеют длину 32 бита, то есть 4 байта.

9.1. Линейная программа

Пример:

```
addi t0, x0, 5
addi t1, x0, 7
add  t2, t0, t1
sw   t2, 0(s0)
```

1. `PC` указывает на первую команду;
2. она выполняется;
3. `PC` увеличивается на 4;
4. выполняется следующая команда;
5. и так далее.

На аппаратном уровне должен существовать путь:

- либо `PC + 4`;
- либо новый адрес перехода/ветвления.

Значит, нужен механизм выбора следующего значения `PC`. Обычно это мультиплексор на входе регистра `PC`.

10. Условные переходы

Если бы программа всегда шла только по схеме `PC = PC + 4`, невозможно было бы реализовать:

- `if`;
- `while`;
- `for`;
- вызовы функций;
- возврат из функций.

Поэтому нужны команды, которые умеют менять линейный ход исполнения программы.

Основные ветвления RV32I

- `beq rs1, rs2, label` — перейти, если равны;
- `bne rs1, rs2, label` — перейти, если не равны;
- `blt rs1, rs2, label` — перейти, если `rs1 < rs2`;
- `bge rs1, rs2, label` — перейти, если `rs1 >= rs2`;
- `bltu`, `bgeu` — беззнаковые варианты.

10.1. Пример работы условного перехода

`beq t0, t1, equal_case`

```
                           Instruction memory

   0x0000   +-----------------------------+   addi t0, x0, 5
   0x0004   +-----------------------------+   addi t1, x0, 5
PC 0x0008   +-----------------------------+   beq  t0, t1, equal_case
   0x000C   +-----------------------------+   addi t2, x0, 0
   0x0010   +-----------------------------+   equal_case: addi t2, x0, 1

                    t0 --------\
                                > compare (==) ---- branch_taken
                    t1 --------/

          PC + 4 -----------------------------------\
                                                     > next PC
          branch target = PC + immediate -----------/

If t0 == t1:
    PC : 0x0008 -> 0x0010

If t0 != t1:
    PC : 0x0008 -> 0x000C
```

11. Безусловный переход

Для безусловного перехода `j skip` часто пишут:

```
j loop
```

Это псевдоинструкция и разворачивается в:

```
jal x0, loop
```

```
                          Instruction memory

   0x0000   +-----------------------------+   addi t0, x0, 5
PC 0x0004   +-----------------------------+   j skip
   0x0008   +-----------------------------+   addi t1, x0, 1
   0x000C   +-----------------------------+   skip: addi t1, x0, 2
```

12. Циклы в RISC-V

Циклы реализуются с помощью условных переходов.

С-код:

```c
int sum = 0;
for (int i = 0; i < 10; i++) {
    sum += i;
}
// sum = 0+1+2+...+9 = 45
```

Пример с массивом:

```c
int sum_array(int *arr, int n) {
    int sum = 0;
    for (int i = 0; i < n; i++)
        sum += arr[i];
    return sum;
}
```

RISC-V ассемблер:

```
# Простой цикл
li t0, 0           # t0 = i = 0
li t1, 0           # t1 = sum = 0
loop:
  bge t0, 10, end  # if i >= 10, выход
  add t1, t1, t0   # sum += i
  addi t0, t0, 1   # i++
  j loop           # вернуться в loop
end:
  # t1 = 45

# Функция sum_array
# a0 = указатель на массив
# a1 = n (количество элементов)
sum_array:
    li t0, 0        # t0 = sum = 0
    li t1, 0        # t1 = i = 0
loop:
    bge t1, a1, end # if i >= n, выход
    lw t2, 0(a0)    # t2 = arr[i]
    add t0, t0, t2  # sum += arr[i]
    addi a0, a0, 4  # сдвиг на следующий элемент
    addi t1, t1, 1  # i++
    j loop
end:
    mv a0, t0       # результат в a0
    jalr x0, 0(ra)
```

13. Введение в процедуры (функции)

Что такое процедура?

Процедура — это набор инструкций, который можно вызвать из разных мест программы.

Основные проблемы процедур:

1. **передача данных** — как передать аргументы?
2. **возврат из процедуры** — куда вернуться?
3. **сохранение состояния регистров** — какие регистры сохранять?
4. **выделение памяти** — где хранить локальные переменные?

13.1. Пример: простейшая функция

C-код:

```
int add(int x, int y) {
    return x + y;
}

// Вызов:
int result = add(10, 20);
```

RISC-V ассемблер:

```
li a0, 10    # аргумент 1
li a1, 20    # аргумент 2
jal ra, add  # вызов, ra=адрес возврата

add:
    add a0, a0, a1  # сумма
    jalr x0, 0(ra)  # возврат
```

14. Инструкции для вызова процедур

Две основные инструкции

- `jal rd, label` — **jump and link**: перейти на `label`, адрес следующей инструкции сохранить в `rd`
- `jalr rd, offset(rs1)` — **jump and link register**: перейти на адрес в `rs1 + offset`, адрес возврата в `rd`

Как работает jal и jalr

```
main:
    jal ra, add_numbers    # ra сохранит адрес следующей строки (0x1008)
    # сюда вернёмся (адрес 0x1008)

add_numbers:               # адрес 0x1004
    add t0, a0, a1
    jalr x0, 0(ra)         # перейти на адрес в ra (0x1008)
```

Ключевые моменты:

- `ra = x1` (return address register)
- `jal` **автоматически** сохраняет адрес возврата
- `jalr` выполняет возврат **туда, где сохранён** `ra`

14.1. Возврат результатов функций

Куда записывается результат функции?

- Результат возвращается в регистре `a0` (для 32-битных значений)
- Для 64-битных значений: `a0` (младшие 32 бита) + `a1` (старшие 32 бита)
- **Вызывающая функция** должна **немедленно** использовать или сохранить результат!

Почему результат не теряется?

```
# Правильно:
jal ra, add      # вызвать add - результат в a0
mv t0, a0        # НЕМЕДЛЕННО сохранить результат в t0

# Неправильно:
jal ra, add      # вызвать add - результат в a0
jal ra, sub      # результат add() ТЕРЯЕТСЯ! a0 перезаписан
```

14.2. Вложенные вызовы и сохранение адреса возврата

А если много вложенных вызовов функций?

```
main:
    jal ra, func1     # ra = адрес возврата в main
    # результат func1 будет в a0

func1:
    addi sp, sp, -8   # сохранить ra и результат func2
    sw ra, 4(sp)      # сохранить ra
    jal ra, func2     # ra = адрес возврата в func1
    sw a0, 0(sp)      # сохранить результат func2 в a0
    # ... использовать результат func2 ...
    lw a0, 0(sp)      # восстановить результат func2
    lw ra, 4(sp)      # восстановить ra
    addi sp, sp, 8
    jalr x0, 0(ra)    # возврат в main

func2:
    jal ra, func3     # ещё один уровень вложенности
    # результат func3 в a0, func2 может его обработать
    jalr x0, 0(ra)    # возврат в func1 с результатом в a0
```

Память на стеке на каждый уровень:

- `ra` — 4 байта (обязательно)
- `a0` — 4 байта (если нужен результат после следующего вызова)
- Сохраняемые регистры `s0-s11` — 4 байта каждый
- Локальные переменные — столько, сколько нужно

15. Регистры в RISC-V: роли и сохранение

15.0. Зачем нужна система ролей и сохранения?

Проблема: как разработчик может быть уверен, что функция не испортит значения в регистрах?

Без правил:

```
main:
    li s1, 100       # сохранили значение в s1
    jal ra, mystery  # вызвали неизвестную функцию
    add t0, s1, 1    # S1 изменился?! ОШИБКА...
```

Решение: установить **соглашение о вызовах** (calling convention) — набор правил, которые обязательны для всех функций:

- **Временные регистры** (`t0-t6`): вызванная функция **может** менять без ограничений
- **Сохраняемые регистры** (`s0-s11`): вызванная функция **обязана** восстановить после использования
- **Аргументы и результаты**: всегда в фиксированных регистрах (`a0-a7`)

Выгода:

1. Разработчик может вызвать незнакомую функцию и быть уверен в безопасности `s0-s11`
2. Функции могут работать независимо (в любом порядке, из разных мест)
3. Компилятор знает, как оптимально распределять регистры

15. Регистры в RISC-V: роли и сохранение

Регистр	ABI	Назначение	Кто сохраняет
`x0`	`zero`	Константа 0	—
`x1`	`ra`	Адрес возврата	Вызывающая
`x2`	`sp`	Указатель стека	Вызванная
`x5-x7`	`t0-t2`	Временные	Вызывающая*
`x8-x9`	`s0-s1`	Сохраняемые	**Вызванная**
`x10-x17`	`a0-a7`	Аргументы, результаты	Вызывающая
`x18-x27`	`s2-s11`	Сохраняемые	**Вызванная**
`x28-x31`	`t3-t6`	Временные	Вызывающая*

*Вызывающая функция может потерять эти значения!

Основное правило: если функция использует `s0` - `s11`, она обязана их сохранить/восстановить.

15.1. Пример: различие между временными и сохраняемыми регистрами

```
main:
    li s0, 100        # используем сохраняемый регистр
    li t0, 200        # используем временный регистр

    jal ra, modify    # вызов

    # После возврата:
    # s0 ещё = 100  (modify сохранил его!)
    # t0 уже НЕ = 200  (временный - потеря!)

modify:
    # Спасаем s0, так как собираемся его менять
    addi sp, sp, -4
    sw s0, 0(sp)

    li s0, 999        # меняем s0
    li t0, 777        # меняем t0 (вызывающая потеряет!)

    lw s0, 0(sp)      # восстанавливаем s0
    addi sp, sp, 4
    jalr x0, 0(ra)
```

Почему `s0` = 100, а `t0` потеряна?

- `s0` — сохраняемый регистр. `modify` спасла его скопировав на стек, и восстановила после своей работы.
- `t0` — временный регистр. `modify` может его менять. Вызывающая (`main`) потеряла его значение.

16. Передача аргументов функции

Согласно соглашению о вызовах RISC-V:

- первые **8 целочисленных аргументов** — в регистрах `a0-a7`;
- остальные — на стеке;
- **результат** возвращается в `a0` (и `a1` для 64-битных значений).

Пример: функция с несколькими аргументами

```
int multiply_add(int x, int y, int z) {
    return x * y + z;
}
```

Вызов: `multiply_add(3, 4, 5)` должна вернуть `3*4 + 5 = 17`

```
li a0, 3           # первый аргумент x
li a1, 4           # второй аргумент y
li a2, 5           # третий аргумент z
jal ra, multiply_add

# Внутри multiply_add:
multiply_add:
    mul t0, a0, a1 # t0 = x * y
    add a0, t0, a2 # a0 = t0 + z (результат в a0)
    jalr x0, 0(ra)
```

17. Классификация регистров: временные vs сохраняемые

Два типа регистров

Временные (Caller-saved) — `t0-t6`

- Вызывающая функция **должна** сохранить перед вызовом, если нужны после
- Вызванная функция может их менять без предупреждения

Сохраняемые (Callee-saved) — `s0-s11`

- Вызванная функция **обязана** сохранить/восстановить
- Вызывающая функция может использовать их

```
# Функция НЕ использует s0 → не нужно спасать
simple_func:
    li t0, 100    # временный - вызывающая потеряет
    jalr x0, 0(ra)

# Функция ИСПОЛЬЗУЕТ s0 → нужно спасать
complex_func:
    addi sp, sp, -4
    sw s0, 0(sp)   # обязательно!
    li s0, 100     # используем s0
    # работа
    lw s0, 0(sp)   # восстанавливаем
    addi sp, sp, 4
    jalr x0, 0(ra)
```

18. Стек (Stack) — концепция

Стек — это особая область памяти для:

- **сохранения адреса возврата** при вложенных вызовах;
- **локальных переменных** функции;
- **сохранённых регистров** (если функция их использует).

Свойства стека в RISC-V

- Растёт **вниз** (в сторону уменьшения адресов)
- `sp` указывает на **верхушку** (текущая позиция)
- **push**: `addi sp, sp, -4; sw reg, 0(sp)`
- **pop**: `lw reg, 0(sp); addi sp, sp, 4`

```
Адреса памяти

0x7000_0000  +---------+
             |  старые |
             |  данные |
             +---------+ ← sp (верхушка стека)
             |  новые  |  (здесь пишем)
             +---------+
0x1000_0000  |   heap  |
```

Вложенные вызовы используют стек для цепочки адресов возврата.

19. Работа со стеком в RISC-V

Как использовать стек для сохранения регистров

```
# Сохранить значения на стеке
addi sp, sp, -8        # выделить 8 байт
sw t0, 0(sp)           # сохранить t0
sw t1, 4(sp)           # сохранить t1

# ... работа ...

# Восстановить значения со стека
lw t1, 4(sp)           # восстановить t1
lw t0, 0(sp)           # восстановить t0
addi sp, sp, 8         # освободить память
```

Цепочка вложенных вызовов

```
main()
  ↓ jal ra, func1    (ra → 0x1008, sp указывает на адрес возврата)
  func1()
    ↓ jal ra, func2  (ra → 0x2008, перед этим func1 спасла свой ra!)
    func2()
      ↓ выполнение
    ← jalr возврат в func1
  ← jalr возврат в main
```

20. Локальные переменные на стеке

Как хранить локальные переменные

```
int calculate(int x, int y) {
    int a = x + 5;           // локальная переменная
    int b = y - 3;           // локальная переменная
    return a + b;
}
```

На RISC-V:

```
calculate:
    # a0 = x, a1 = y

    addi sp, sp, -8        # выделить место для a и b, sp+0 = b, sp+4 = a

    addi t0, a0, 5         # t0 = x + 5
    sw t0, 4(sp)           # a = t0 (сохранить по адресу sp+4)

    addi t1, a1, -3        # t1 = y - 3
    sw t1, 0(sp)           # b = t1 (сохранить по адресу sp+0)

    lw t2, 4(sp)           # t2 = a
    lw t3, 0(sp)           # t3 = b
    add a0, t2, t3         # a0 = a + b (результат)

    addi sp, sp, 8         # освободить память
    jalr x0, 0(ra)
```

21. Соглашение о вызовах (Calling Convention) в RISC-V

Соглашение описывает, какие регистры используют, кто их сохраняет, и как передаются аргументы. Рассмотрим пример вызова функции

Вызывающая сторона (caller):

```
# 1. Подготовить аргументы
li a0, 10
li a1, 20
# 2. Вызвать
jal ra, my_function
# 3. Результат теперь в a0
```

Вызываемая функция (callee):

```
my_function:
    # 1. Если используем s0-s11, сохраняем их
    addi sp, sp, -4
    sw s0, 0(sp)

    # 2. Выделяем место для локальных переменных (если нужны)
    addi sp, sp, -8

    # 3. Работаем: используем a0, a1, наши локальные переменные
    add s0, a0, a1     # s0 = a0 + a1
    sw s0, 0(sp)       # сохраняем результат на стеке

    # 4. Возвращаем результат в a0
    mv a0, s0

    # 5. Восстанавливаем sp и сохраненные регистры
    addi sp, sp, 8     # освобождаем локальные переменные
    lw s0, 0(sp)       # восстанавливаем s0
    addi sp, sp, 4

    # 6. Возвращаемся
    jalr x0, 0(ra)
```

22. Пример: функция с сохранением регистров

```c
// С-код
int sum_array(int *arr, int n) {
    int sum = 0;
    for (int i = 0; i < n; i++)
        sum += arr[i];
    return sum;
}
```

```
# RISC-V реализация
# a0 = адрес массива (int *)
# a1 = n (количество элементов)

sum_array:
    # Спасаем s0 и s1 (сохраняемые регистры!)
    addi sp, sp, -8
    sw s0, 0(sp)
    sw s1, 4(sp)

    li s0, 0           # s0 = sum
    li s1, 0           # s1 = i

loop:
    bge s1, a1, done   # if i >= n, выход
    lw t0, 0(a0)       # t0 = arr[i]
    add s0, s0, t0     # sum += arr[i]
    addi a0, a0, 4     # сдвиг на следующий элемент
    addi s1, s1, 1     # i++
    j loop

done:
    mv a0, s0          # результат в a0

    # Восстанавливаем регистры
    lw s1, 4(sp)
    lw s0, 0(sp)
    addi sp, sp, 8
    jalr x0, 0(ra)
```

23. Пример: функция с временными регистрами

Простая функция, использующая только временные регистры (без сохранения).

```
# Функция умножения через повторное сложение
# a0 = x, a1 = y
# Вернуть x * y в a0

multiply:
    li t0, 0           # t0 = результат
    li t1, 0           # t1 = счётчик

loop_mul:
    bge t1, a1, done_mul   # if counter >= y, выход
    add t0, t0, a0         # результат += x
    addi t1, t1, 1         # counter++
    j loop_mul

done_mul:
    mv a0, t0          # результат в a0
    jalr x0, 0(ra)
```

Вызывающая функция **может потерять** t0, t1 после вызова!

```
# В главной программе:
li t0, 100         # t0 = 100
jal ra, multiply   # вызов multiply
# t0 уже НЕ = 100! (временный регистр)
```

25. Передача больших структур

Для больших данных не используются регистры, а только указатели.

```
struct Point { int x, y; };
struct Matrix { int data[100]; };

void transform(struct Point p);      // 8 байт → может в регистрах
void process(struct Matrix m);       // 400 байт → только указатель!
```

Правило: если структура > 16 байт, передаётся **указатель** на неё.

```
# Вызов: передаём указатель на структуру
la a0, my_matrix       # a0 = адрес структуры
jal ra, process

# Внутри process:
process:
    # a0 = указатель на Matrix
    lw t0, 0(a0)       # t0 = data[0]
    lw t1, 4(a0)       # t1 = data[1]
    # работа
    sw t0, 0(a0)       # сохраняем изменения
    jalr x0, 0(ra)
```

Преимущество: быстро (копируем только адрес), экономно по стеку.

26. Когда НЕ стоит использовать указатели

Указатели добавляют разыменование (замедление и усложнение кода).

Когда избежать указателей:

- **Малые структуры** (≤ 8 байт) — передать значения в регистрах
- **Двумерные массивы** — часто быстрее передать непосредственно
- **Простые данные** — int, float — не нужны указатели

Плохой пример (много разыменований):

```
# Плохо: работаем через указатели
process:
    lw t0, 0(a0)       # разыменование #1
    lw t1, 4(a0)       # разыменование #2
    add t2, t0, t1
    sw t2, 8(a0)       # разыменование #3
```

Пример лучше (прямые аргументы):

```
# Хорошо: аргументы в регистрах
process:
    add a2, a0, a1     # a0, a1, a2 - прямые значения
    jalr x0, 0(ra)
```

Когда указатели НЕОБХОДИМЫ:

- **Большие массивы** (> 1000 элементов)
- **Вывод множественных значений** (через outparam)
- **Динамические данные** (malloc, размер неизвестен)

27. Карта памяти программы

Как разделяется память для типичной программы на C.

```
Адреса памяти

Высокие
            ┌──────────────────┐
            │ переменные окруж.│
            ├──────────────────┤
            │                  │
            │  STACK (растёт ↓)│
            │                  │
            │ (свободное место)│
            │                  │
            ├──────────────────┤
            │ HEAP (растёт ↑)  │   malloc, new
            ├──────────────────┤
            │ .bss             │   неинициализированные данные
            │ int arr[100];    │
            ├──────────────────┤
            │ .data            │   инициализированные данные
            │ int x = 10;      │
            ├──────────────────┤
            │ .text            │   машинный код
            │ (инструкции)     │
Низкие      └──────────────────┘
```

28. Карта памяти RISC-V

Типичная архитектура памяти для 32-битной системы.

Раздел	Адреса	Использование
Stack	0x7FFF_FFFF — ...	Локальные переменные функ., адреса возврата
Heap	0x1000_0000 — ...	malloc(), new, динамическая память
BSS	0x0804_0000 — ...	`static int arr[1000];` (нулевое значение)
Data	0x0800_0000 — ...	`int x = 10;` (инициализированные глобалные)
Text	0x0040_0000 — 0x...	Машинный код программы (read-only)

Регистр gp (Global Pointer)

Для быстрого доступа к глобальным данным:

```
lw t0, offset(gp)     # прочитать глобальную переменную
  # эквивалентно: lw t0, &global_var
```

Компилятор инициализирует `gp` на точку в `.data` сегменте.

29. Структура скомпилированного файла (ELF)

Исполняемый файл содержит множество сегментов и информации.

Из чего состоит ELF файл:

- **ELF header** — метаданные файла (архитектура, точка входа)
- **.text segment** — машинный код (read-only)
- **.data segment** — инициализированные глобальные данные (read-write)
- **.bss segment** — неинициализированные данные (только размер, экономия места)
- **symbol table** — имена функций, переменных (для отладчика)
- **relocation table** — как соединять разные объектные файлы
- **debug info** — информация для отладчика

Загрузчик ОС:

При запуске программы загрузчик:

1. Читает ELF header
2. Создаёт процесс в памяти
3. Копирует .text в память
4. Копирует .data в память
5. Выделяет место под .bss (заполняет нулями)
6. Подготавливает стек
7. Передаёт управление на `_start` или `main`

30. Процесс компиляции программы на RISC-V

Как код C становится исполняемым файлом.

```
main.c                          library.c
   ↓                               ↓
 ┌──────────────────────┐        ┌──────────────────┐
 │ Препроцессор         │        │ Препроцессор     │
 │ (замена макросов)    │        │                  │
 └──────────────────────┘        └──────────────────┘
   ↓                               ↓
 ┌──────────────────────┐        ┌──────────────────┐
 │ Компилятор           │        │ Компилятор       │
 │ (C → RISC-V ассм)    │        │                  │
 └──────────────────────┘        └──────────────────┘
   ↓                               ↓
 ┌──────────────────────┐        ┌──────────────────┐
 │ Ассемблер            │        │ Ассемблер        │
 │ (ассм → машинный)    │        │                  │
 └──────────────────────┘        └──────────────────┘
   ↓                               ↓
main.o                          library.o
   │                               │
   └───────────────┬───────────────┘
                   ↓
         ┌──────────────────────────┐
         │ Компоновщик (linker)     │
         │ (объединяет .o файлы,    │
         │  разрешает ссылки)       │
         └──────────────────────────┘
                   ↓
              a.out (ELF)
                   ↓
         ┌──────────────────────────┐
         │ Загрузчик ОС             │
         │ (копирование в память)   │
         └──────────────────────────┘
                   ↓
        процесс в оперативной памяти
```

30.1 Процесс компиляции программы на RISC-V(1)

На каждом этапе:

1. **Препроцессор** (`cpp`): `#include` , `#define`
2. **Компилятор** (`gcc -S`): C → RISC-V ассемблер
3. **Ассемблер** (`as`): ассемблер → машинный код (`.o`)
4. **Компоновщик** (`ld`): связать .o файлы, разрешить external references(call func_name)
5. **Загрузчик**: поместить программу в памят и запустить